Handle document generation logic in routes.py

Slide 1 – Document Generation Logic

Handle Document Generation Logic in `routes.py`

- `routes.py` manages document-generation API requests.
- It receives data from the frontend.
- It validates the input using Pydantic.
- It calls the Gemini document generator.
- It returns the generated legal document.

Slide 2 – Pydantic Model

Define `DocumentRequest`

- Pydantic is used for request validation.
- `DocumentRequest` defines the required input fields.
- It ensures the received data has the correct structure.
- Main fields:
    - `document_type`
    - `parties`
    - `terms`
    - `dates`

Slide 3 – `DocumentRequest` Structure

Request Data Model

```
from pydantic import BaseModel

class DocumentRequest(BaseModel):
    document_type: str
    parties: str
    terms: str
    dates: str
```

Slide 4 – Generate API Route

Create `/generate` Endpoint

- Define a POST endpoint for document generation.
- Receive `DocumentRequest` as input.
- Extract the required document details.
- Pass the information to the generator.

```
@router.post("/generate")
def generate_document(request: DocumentRequest):
    # Process request
```

Slide 5 – Gemini Document Generator

Call `generate_document()`

- Create or access the `GeminiDocumentGenerator`.
- Call `generate_document(...)`.
- Pass the document type, parties, terms, and dates.
- Gemini processes the information and generates the document text.

```
result = generator.generate_document(
    request.document_type,
    request.parties,
    request.terms,
    request.dates
)
```

Slide 6 – Response & Workflow

Complete Document Generation Flow

Frontend → `routes.py` → Pydantic Validation → GeminiDocumentGenerator → Generated Document

- User submits document requirements.
- `Document Request` validates the data.
- `generate document()` sends the information to Gemini.
- Generated legal text is returned to the frontend.
- The user can review and use the generated document.